

# CAP Theorem & PACELC

Phase 2 · Intermediate · Estimated time: 1.5 hours

---

## 1. Concept From First Principles

The CAP theorem, formalized by Eric Brewer in 2000, is probably the single most name-dropped idea in system design interviews — and also one of the most frequently misused, because candidates often quote it without understanding exactly what it does and doesn't claim. Understanding it precisely, using the consistency and availability vocabulary from Chapters 2 and 3, will put you ahead of most candidates who've only memorized the letters.

CAP states that a distributed data store can provide at most two of the following three guarantees simultaneously: **Consistency** (every read receives the most recent write, or an error — the strong consistency from Chapter 3), **Availability** (every request to a non-failing node receives a response, even if it might be stale), and **Partition tolerance** (the system keeps operating even when network messages between nodes are dropped or delayed). The critical, commonly-missed detail: in any real distributed system running across multiple machines or data centers, network partitions *will* happen — a cable gets cut, a router fails, a data center loses connectivity. Partition tolerance isn't really a choice you get to opt out of; it's a fact of operating at scale. So CAP, in practice, reduces to a choice between C and A specifically *during* a partition.

A **CP** system, when a partition happens, chooses consistency: it will refuse to serve some requests (returning an error, or simply timing out) rather than risk returning stale or conflicting data. A **AP** system chooses availability: it keeps answering every request, even from nodes that may be cut off from the rest of the cluster and therefore might be serving stale data. Note carefully what CAP does *not* say: it says nothing about system behavior when there is *no* partition, which is the vast majority of the time for most systems. That's precisely the gap PACELC was created to fill.

### Backend engineer analogy

Think about what your Laravel API does when it can't reach its database at all — a connection timeout. Do you return a 503 and tell the user to retry (choosing consistency: never guess, never serve wrong data), or do you serve a cached, possibly-stale response so the user sees *something* (choosing availability)? Every timeout/fallback decision you've ever coded is a small, local instance of the exact C-vs-A choice CAP describes at the level of an entire distributed database.

## PACELC: what happens when there's no partition

PACELC extends CAP with the observation that even in the normal, no-partition case, distributed systems still face a trade-off: if there's no Partition (P), then Else (E), you must choose between Latency (L) and Consistency (C). Making every read/write wait for a majority of replicas to confirm gives strong consistency but adds latency to every single operation, partition or not; skipping that coordination gives lower latency at the cost of some consistency, permanently, not just during rare outages. This is why PACELC is considered the more practically useful framework day-to-day — partitions are rare events, but the latency-versus-consistency trade-off is live on literally every request.

In PACELC terms, a system like DynamoDB or Cassandra, configured for low-latency reads, is typically PA/EL — available during a partition, and choosing low latency over strict consistency the rest of the time. A system like a traditional single-primary relational database with synchronous replication tends toward PC/EC — consistent during a partition (it will refuse writes rather than risk divergence) and consistent the rest of the time too, accepting the latency cost. Neither is universally "better" — the correct classification depends entirely on what the specific data actually needs, echoing the per-field reasoning from Chapter 3.

A final nuance worth stating explicitly, since it trips up many candidates: CAP is about the behavior of a single, specific data store or service during a network partition, not a label you can slap on an entire multi-service architecture. A well-designed system often mixes CP components (a payments ledger) and AP components (a product recommendation cache) side by side, exactly as Chapter 3 argued for mixing consistency models — CAP is the theoretical grounding for why that mixing is not just allowed but often the correct design.

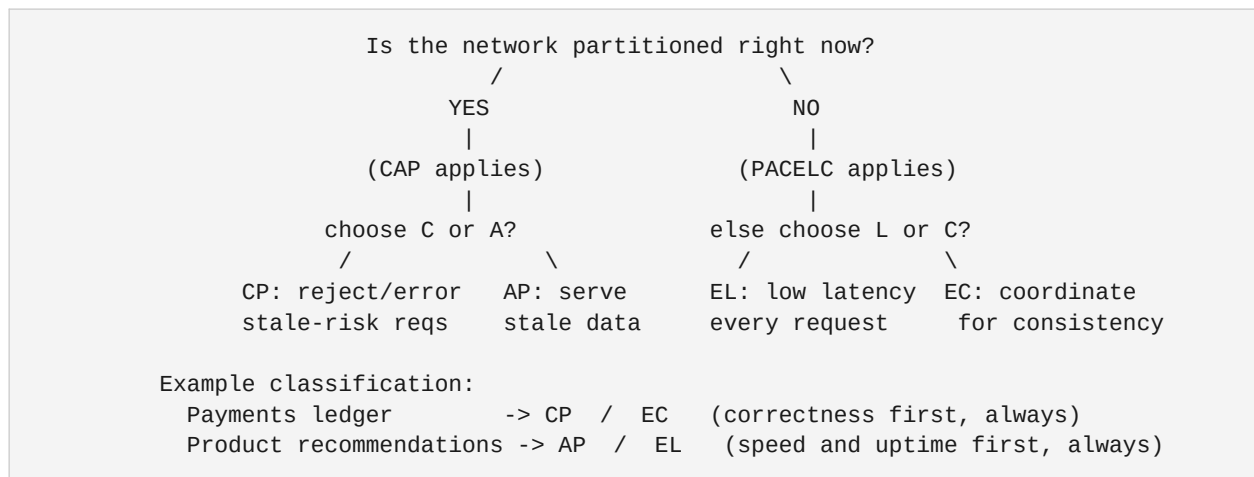
## 2. Real-World Example / Case Study

Razorpay's and PhonePe's core transaction-processing systems lean CP: during a network partition between data centers, they would rather reject or delay a payment than risk two conflicting versions of the same transaction being confirmed independently on both sides of the partition. A rejected-and-retried payment is an inconvenience; a double-processed or lost payment is a serious financial and regulatory problem.

Contrast that with Swiggy's or Talabat's restaurant-search and menu-browsing systems, which lean AP: if a partition isolates one data center, it's far better for that region to keep serving search results (possibly built from slightly stale menu data) than to show users an error page while they're trying to order dinner. A stale "currently open" flag on a restaurant is a minor annoyance; a blank search page during peak dinner hours is lost revenue and a genuinely bad user experience.

Noon and Flipkart's inventory-count displays are a good PACELC illustration in the no-partition case: showing an exactly-accurate "3 left in stock" count on every single page load would require checking with the authoritative inventory system on every request, adding latency to every product page visit. Instead, many e-commerce platforms show a cached, slightly-stale count (EL: favoring latency) and only enforce a strongly consistent check (EC) at the final "place order" step, where correctness actually matters.

## 3. Diagram



## 4. Trade-offs Table

| System type                   | CAP choice | PACELC choice | Example                                  |
|-------------------------------|------------|---------------|------------------------------------------|
| Payments / ledger             | CP         | EC            | Razorpay/PhonePe transaction core        |
| Search / catalog browsing     | AP         | EL            | Swiggy/Talabat restaurant search         |
| Inventory display (page load) | AP         | EL            | Product listing 'in stock' count         |
| Inventory check (checkout)    | CP         | EC            | Final stock reservation before payment   |
| Session / cache store         | AP         | EL            | Redis-backed session or view-count cache |

## 5. Common Interview Questions

### Q1: "Explain CAP theorem and why you can't have all three."

Model approach: Define all three terms precisely, then explain that partition tolerance isn't optional in a real distributed system (partitions will happen), so the practical choice is between consistency and availability specifically during a partition — not a free three-way pick.

### Q2: "Why is PACELC considered more useful than CAP for day-to-day design decisions?"

Model approach: Point out that partitions are rare, but the latency-vs-consistency trade-off PACELC describes is present on every single request, partition or not — so PACELC captures the trade-off engineers actually navigate constantly, while CAP only describes rare failure-mode behavior.

### Q3: "Would you classify your payments system as CP or AP? Justify it."

Model approach: CP — justify by weighing the cost of unavailability (a rejected request, retried moments later) against the cost of inconsistency (a lost or duplicated financial transaction), and explicitly state that the second cost is unacceptable for this domain.

### Q4: "Can a single system be CP for some data and AP for other data?"

Model approach: Yes, and good systems usually are — CAP classifies a specific data store or service, not an entire architecture. Give a concrete example: an e-commerce platform's order/payment core is CP, while its recommendation and search services are AP, deliberately, based on what each actually needs.

## 6. Practice Exercises

- 1 Classify each of the following as CP or AP, with a one-sentence justification: a chat app's message-delivery service, a hotel-booking system's room-availability check, a website's page-view counter.
- 2 Explain, using PACELC, why a single-region relational database with synchronous replication tends to be PC/EC, while a globally distributed NoSQL store tuned for speed tends to be PA/EL.
- 3 Describe a scenario at a ride-hailing company (like Careem) where choosing AP over CP for the wrong piece of data (e.g., trip fare calculation) could cause a real business problem.

## 7. Curated YouTube Videos

| Language | Video & Channel                                                       | Why it helps                                                         |
|----------|-----------------------------------------------------------------------|----------------------------------------------------------------------|
| English  | "CAP Theorem Explained in 5 minutes   System Design Interview Basics" | Fast, precise definition-level pass, good for first exposure.        |
| English  | "PACELC Theorem   Beyond CAP Theorem   System Design Primer"          | Directly covers the PACELC extension this chapter emphasizes.        |
| Hindi    | "CAP Theorem    System Design    Hindi"                               | Full Hindi explanation with the classic C/A/P trade-off walkthrough. |
| Hindi    | "CAP Theorem Simplified 2026" — Scaler (Anshuman Singh)               | Hinglish explanation from an India-based system design educator.     |

## 8. Key Takeaways

- CAP theorem: a distributed system can give at most 2 of Consistency, Availability, Partition tolerance.
- Partition tolerance is effectively mandatory in real systems, so the real choice is C vs. A during a partition.
- PACELC extends CAP: even without a partition, you still trade Latency vs. Consistency on every request.
- CP systems reject/delay requests during a partition rather than risk inconsistency; AP systems keep answering.
- CAP/PACELC classification applies per data store or service, not to an entire system as one label.
- Financial/ledger data almost always leans CP/EC; browsing/search/counters almost always lean AP/EL.
- Interviewers value you justifying the CP/AP choice with concrete business cost, not just naming the letters.